

University of the Philippines Diliman

Department of Computer Science

CS 171: Parallel Computing — Mini-Project Final Report

**Parallel Modular Exponentiation:
Task-Level GPU Parallelism versus
Intra-Message Bit-Parallel Precomputation**

Ken Duculan Mark Cuizon

University of the Philippines Diliman, Quezon City, Philippines

June 5, 2026

Abstract

Modular exponentiation ($b^e \bmod m$) is the computational core of RSA encryption, yet its inherent sequential dependency chain — each squaring step depends on the previous — makes GPU parallelisation non-trivial within a single operation. This paper examines two strategies for accelerating modular exponentiation on a laptop-class NVIDIA RTX 3050 GPU against a GMP-based sequential CPU baseline. The first strategy, *GPU Batch*, exploits task-level parallelism: n independent messages are dispatched simultaneously, each handled by one CGBN warp executing one square-and-multiply chain. The second, *Bit-Parallel*, attempts to break the intra-message dependency by precomputing all powers $b^{2^i} \bmod m$ in parallel before accumulating selected powers in a sequential scan. GPU Batch achieves consistent speedups scaling with both batch size and operand bit-width, peaking at **30.5×** for 1024-bit inputs at $N = 1,000$. Bit-Parallel produces correct results but underperforms the sequential baseline at all operand widths of 128 bits and above across every tested batch size. At narrower widths, GPU parallelism briefly overcomes GMP’s adaptive-precision advantage for small N , though the margin inverts as batch size grows. At 1024-bit operands the deficit compounds rapidly: Bit-Parallel trails GPU Batch by $380\times$ at $N = 1,000$, driven by a quadratic Phase 1 work blowup, roughly 50% average warp idleness, and an Amdahl ceiling that provably caps speedup over an equal-precision sequential baseline below $0.75\times$ regardless of Phase 1 parallelism. All GPU measurements are kernel-only; PCIe transfer overhead is excluded. The results demonstrate that task-level parallelism is the correct granularity for this workload, while the Bit-Parallel restructuring increases total arithmetic by $O(\text{ebits}/2)$ — a blowup the available GPU resources cannot offset at practical scales.

1. Introduction

Modular exponentiation — computing $b^e \bmod m$ for large integers — is a fundamental operation in RSA public-key cryptography. RSA-based key exchange, digital signatures, and certificate verification all rely heavily on modular exponentiation, making it an important target for acceleration in secure computing systems.

The central algorithmic challenge is the presence of strong data dependencies. Binary square-and-multiply processes exponent e bit by bit: it squares the running result at every step and conditionally multiplies by the base when the current bit is 1. Step k depends on the result produced by step $k - 1$, forming a chain of $O(\log e)$ dependent operations. These dependencies severely limit opportunities for efficient parallelisation within a single exponentiation.

Two responses are natural. The first ignores intra-message dependencies and instead parallelises across independent messages, assigning one GPU warp to each modular exponentiation and processing many exponentiations simultaneously. The second attempts to restructure a single exponentiation to expose intra-message parallelism through precomputation of intermediate powers.

This paper implements both strategies using the Cooperative Groups Big Number (CGBN) library, benchmarks them against a GMP-based sequential baseline across operand sizes from 32 to 1024 bits, and evaluates their behaviour using work efficiency, Amdahl’s Law, thread divergence, and compute-to-memory access ratios. The results demonstrate a fundamental principle of parallel computing: exposing parallelism alone is insufficient if doing so introduces more work than the available hardware can recover through concurrent execution.

2. Background

2.1. Binary Modular Exponentiation

Given base b , exponent e , and modulus m , the goal is to compute $b^e \bmod m$. A naïve implementation performs e multiplications, which becomes infeasible for cryptographic operand sizes. Binary exponentiation reduces this cost to $O(\log e)$ by processing the exponent one bit at a time: the current result is squared at every iteration and multiplied by b only when the current bit is 1.

In the benchmark datasets used in this project, exponent sizes scale with operand bit-width. Consequently, a 1024-bit instance requires approximately 1024 squar-

ing steps and up to 1024 additional multiplications. Since each operation involves multi-precision arithmetic on large integers, the computational cost per message grows substantially with operand size.

2.2. GPU Execution Model and CGBN

NVIDIA GPUs execute threads in groups of 32 called *warps*. Threads within a warp execute in lockstep according to the Single-Instruction Multiple-Thread (SIMT) model. When threads in a warp follow different execution paths, the warp experiences *thread divergence*, forcing the hardware to serialise the divergent branches and reducing efficiency.

The RTX 3050 Laptop GPU used in this project contains 16 Streaming Multiprocessors (SMs) and 2,048 CUDA cores. Multiple warps can reside on each SM simultaneously, allowing the hardware scheduler to switch between warps while others wait on arithmetic or memory operations. This ability to maintain many active warps is critical for achieving high throughput.

Implementing large-integer arithmetic efficiently on a GPU presents a partitioning problem. A 1024-bit integer consists of 32 limbs of 32 bits each. The CGBN library addresses this by assigning one limb to each thread in a warp, using $\text{TPI} = 32$ (*threads per instance*). Threads cooperate through warp-shuffle intrinsics to exchange limb values and perform multi-precision arithmetic collectively. As a result, one 1024-bit CGBN instance maps naturally onto exactly one GPU warp.

This mapping directly motivates the GPU Batch implementation used in this work. Each modular exponentiation is assigned to a single warp, allowing many exponentiations to execute concurrently without communication between instances.

A key constraint of CGBN is that `cgbn_modular_power` requires an odd modulus. This requirement is automatically satisfied for RSA, since the modulus is the product of two odd primes.

2.3. Fixed-Width Arithmetic and Occupancy Effects

All GPU implementations in this project use CGBN configured for a fixed precision of 1024 bits. Consequently, smaller operands are zero-padded and processed using the same arithmetic pipeline as true 1024-bit values. A 64-bit operand therefore incurs the cost of 1024-bit arithmetic, resulting in substantial excess computation relative to an adaptive-width implementation.

In contrast, the GMP baseline dynamically adapts its internal representation to the actual operand size. This gives the CPU implementation a natural efficiency

advantage for narrow operand widths, while the relative overhead of fixed-width GPU arithmetic decreases as operands approach 1024 bits.

Fixed-width arithmetic is not the only factor influencing performance. GPUs achieve high throughput only when enough independent warps are available to occupy all Streaming Multiprocessors and hide arithmetic latency. Small batch sizes provide too few concurrent exponentiations to fully utilise the hardware, leaving many execution resources idle. As the batch size increases, occupancy improves and the GPU becomes increasingly effective at amortising arithmetic latency across many active warps.

Finally, all reported GPU timings measure kernel execution time only. Host-device memory transfers, file I/O, and other external overheads are excluded from the measurements. Consequently, observed performance trends primarily reflect computational behaviour inside the GPU kernel, including occupancy, arithmetic intensity, memory bandwidth utilisation, and algorithmic work complexity.

3. Algorithms and Design

3.1. Sequential Baseline

Algorithm 1 Square-and-Multiply (Sequential)

Require: Base b , exponent e , modulus m

Ensure: $b^e \bmod m$

```

1: Reduce  $b \leftarrow b \bmod m$ 
2: result  $\leftarrow 1$ 
3: for each bit of  $e$ , most significant to least significant
   do
4:   result  $\leftarrow \text{result}^2 \bmod m$ 
5:   if current bit = 1 then
6:     result  $\leftarrow \text{result} \times b \bmod m$ 
7:   end if
8: end for return result

```

Implemented using GMP's `mpz_powm`, which handles arbitrary-precision arithmetic and adapts internally to operand size. This serves as the correctness oracle and timing baseline for all speedup calculations. Sequential timings include file I/O within the measured region.

3.2. GPU Batch

Algorithm 2 GPU Batch (Task-Parallel)

Require: Arrays Base[1.. n], Exp[1.. n], Mod[1.. n]

Ensure: Array Result[1.. n]

```

1: parallel for instance  $k = 1$  to  $n$  do
2:    $b \leftarrow \text{Base}[k] \bmod \text{Mod}[k]$ 
3:   result  $\leftarrow 1$ 
4:   for each bit of Exp[ $k$ ], MSB to LSB do
5:     result  $\leftarrow \text{result}^2 \bmod \text{Mod}[k]$ 
6:     if bit = 1 then
7:       result  $\leftarrow \text{result} \times b \bmod \text{Mod}[k]$ 
8:     end if
9:   end for
10:  Result[ $k$ ]  $\leftarrow$  result
11: end parallel for

```

Each of the n tasks maps to one CGBN instance (one GPU warp). The square-and-multiply chain remains sequential *within* each warp, but all n warps execute concurrently. There is zero inter-instance communication — this is a trivially parallel decomposition. Kernel parameters: TPI = 32, TPB = 128 (four instances per block). Block count scales as $\lceil n \cdot \text{TPI} / \text{TPB} \rceil$.

Note: Reported runtimes are kernel-only. Host $\rightarrow$ device and device $\rightarrow$ host PCIe transfers are excluded.

3.3. Bit-Parallel

Algorithm 3 Bit-Parallel Modular Exponentiation

Require: Base b , exponent e , modulus n , bit-width ebits

Ensure: $b^e \bmod n$

Phase 1 — Parallel Precomputation:

```

1: parallel for bit position  $i = 0$  to ebits-1 do
2:   Power[ $i$ ]  $\leftarrow b^{2^i} \bmod n$   $\triangleright i$  squarings from base
3: end parallel for

```

Phase 2 — Sequential Accumulation:

```

4: result  $\leftarrow 1$ 
5: for bit position  $i = 0$  to ebits-1 do
6:   if bit  $i$  of  $e = 1$  then
7:     result  $\leftarrow \text{result} \times \text{Power}[i] \bmod n$ 
8:   end if
9: end for return result

```

Phase 1 launches $n \times \text{ebits}$ CGBN instances simultaneously. Instance (t, i) starts from task t 's base and performs i squarings to produce b^{2^i} . All instances are independent. Phase 2 accumulates the result with n instances scanning all bit positions sequentially. All

$n \times$ ebits intermediate powers must reside in global memory simultaneously: $n \times$ ebits $\times$ 128 B.

Note: Reported runtimes are kernel-only; PCIe transfers excluded.

4. Implementation Notes

4.1. Precision and Overflow Safety

The sequential baseline uses GMP’s arbitrary-precision `mpz_t` type; no overflow is possible and precision adapts to actual operand size.

On the GPU, all values use CGBN’s `mem_t`: a 1024-bit value stored as $32 \times$ `uint32_t` limbs. Inputs of any size are converted via `mpz_export` and zero-padded to fill all 32 limbs. Wide multiplications produce 2048-bit intermediates handled by `cgbn_mul_wide` and `cgbn_rem_wide`. Overflow is structurally impossible.

4.2. Dataset

All test inputs are valid RSA triples (m, e, n) : modulus $n = pq$ generated by Miller-Rabin prime testing; exponent e verified coprime to $\phi(n) = (p-1)(q-1)$; plaintext m coprime to n and validated by encrypt-decrypt round-trip. Since $n = pq$ with odd primes, CGBN’s odd-modulus requirement is automatically satisfied. A fixed seed ensures reproducibility.

4.3. Timing Methodology

GPU timing uses CUDA Events bracketing only the kernel call:

```
1 cudaEventRecord(start);
2 batch_kernel<<<blocks,TPB>>>(
3     d_tasks, d_results, n);
4 cudaEventRecord(stop);
5 cudaEventSynchronize(stop);
6 cudaEventElapsedTime(&ms, start, stop);
```

Listing 1: Kernel-only timing with CUDA Events

PCIe host-to-device and device-to-host transfers are deliberately excluded. The sequential timer brackets the computation loop including file reads and writes. This means sequential times at very small N include file-open latency, slightly overstating GPU speedup at the smallest batch sizes. At large N , I/O is a negligible fraction of sequential time.

Results are averages over ≥ 3 measured runs after one discarded warm-up.

5. Experimental Setup

Table 1: Hardware and Software Specifications

GPU: NVIDIA RTX 3050 Laptop	
Architecture	Ampere (SM 8.6)
CUDA Cores / SMs	2,048 / 16
Global Memory	4 GB GDDR6
Shared Mem/Block	48 KB
Warp size (TPI)	32
TDP	≤ 80 W
CPU: Intel i7-12700H	
Cores / Threads	14 / 20
Max Boost Clock	4.7 GHz
Software	
OS	Fedora 42
CUDA Toolkit	13.1
Big-int (CPU)	GMP (arbitrary prec.)
Big-int (GPU)	CGBN (fixed 1024-bit)

Test configurations span five bit-widths (32, 64, 128, 512, 1024 bits) and batch sizes from $N = 1$ to $N = 100,000$. All three implementations were tested on identical inputs; GPU outputs were verified against the sequential results file at runtime. All tested cases produced correct results.

6. Results and Discussion

Table 2 lists the raw timings. All GPU times are kernel-only, while sequential baseline times include file I/O. Figure 1 visualises the GPU Batch speedup across every evaluated bit-width and batch size (N). Figure 2 examines the 1024-bit configuration in detail and quantifies the Bit-Parallel performance penalty. Detailed architectural analysis of the underlying root causes is deferred to Section 7.

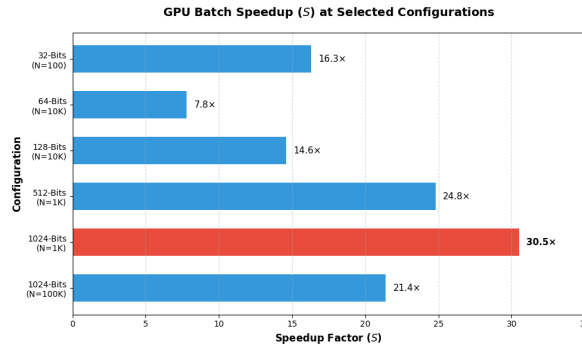


Figure 1: GPU Batch speedup $S = T_{\text{seq}}/T_{\text{GPU}}$ evaluated across selected operand bit-widths and problem sizes (N). Configurations are stacked vertically on the y-axis to compare speedup values along the horizontal x-axis. The 1024-bit ($N = 1\text{K}$) configuration yields the peak performance improvement, highlighted in red at $30.5\times$ speedup, while remaining configurations are shown in blue.

Table 2: Benchmark results at batch sizes $N \in \{1, 100, 1\text{K}, 100\text{K}\}$. *SEQ* indicates the sequential baseline outperforms the GPU; *FAIL* indicates an out-of-memory (OOM) error.

Bits	N	Seq (ms)	GPU (ms)	S	Bit-Par (ms)
32	1	1.203	0.107	11.2×	0.131
	100	2.066	0.127	16.3×	0.507
	1K	6.153	0.622	9.9×	4.029
	100K	419.4	48.19	8.7×	505.1
64	1	0.104	0.291	SEQ	0.303
	100	2.866	0.324	8.8×	2.080
	1K	10.94	1.369	8.0×	18.48
	100K	717.8	108.7	6.6×	2295
128	1	0.147	0.496	SEQ	0.846
	100	4.874	0.731	6.7×	10.62
	1K	34.45	2.926	11.8×	105.2
	100K	1864	262.8	7.1×	11082
512	1	0.456	3.604	SEQ	8.064
	100	37.82	5.161	7.3×	364.4
	1K	501.7	20.20	24.8×	3698
	100K	27551	2201	12.5×	FAIL
1024	1	3.507	11.28	SEQ	31.38
	100	204.5	12.65	16.2×	2495
	1K	1998	65.50	30.5×	24888
	100K	157848	7381	21.4×	FAIL

6.1. Crossover: Minimum Kernel Execution Cost

GPU Batch underperforms the sequential baseline at small batch sizes ($N \leq 10$) for operand widths of 64 bits and above, before crossing over between $N = 10$ and $N = 100$. At 32-bit widths the GPU Batch kernel already outperforms the GMP baseline even at $N = 1$ ($11.2\times$ speedup), as GMP’s cold-start file I/O inflates the sequential timer at that scale; the underlying crossover behaviour at 64-bit and wider inputs is the architecturally representative case. Because the

GPU measurements are strictly kernel-only, this is explicitly an architectural overhead rather than a PCIe transfer bottleneck. The root cause lies in CGBN’s minimum per-kernel execution cost: regardless of how few instances are launched, the GPU must still initialise internal CGBN contexts, load the wide operands, and execute the full square-and-multiply chain at fixed 1024-bit precision.

At $N = 1$ with 64-bit inputs, the GPU kernel (0.291 ms) is significantly slower than the sequential baseline (0.104 ms, which even includes file I/O). This occurs because CGBN’s fixed execution path undergoes fixed 1024-bit word arithmetic, while the GNU Multiple Precision (GMP) library dynamically adapts its precision allocation to the true 64-bit operand width. The crossover point at $N \approx 100$ occurs when a sufficient number of instances are in flight to completely saturate the available streaming multiprocessor (SM) warps, thereby amortising this fixed base kernel overhead.

6.2. Speedup and Bit-Width

Speedup scales positively with operand bit-width, showing a clear, increasing trend visible from 128-bit configurations onward: evaluating to $6.7\times \rightarrow 24.8\times \rightarrow 30.5\times$ at $N = 1\text{K}$ for 128, 512, and 1024 bits respectively. The mechanism behind this trend is rooted in CGBN’s fixed 1024-bit precision framework. When processing narrow 32-bit inputs, CGBN spends substantial hardware cycles computing arithmetic loops scaled for 1024-bit limits.

Conversely, GMP dynamically scales its algorithmic complexity down with smaller operands. Consequently, the sequential baseline’s relative structural advantage is maximized at narrow bit-widths, but completely erodes at 1024 bits where both implementations must execute equivalent total precision arithmetic. The apparent 32-bit speedup anomaly of $16.3\times$ at $N = 100$ occurs because cold-start file I/O skew dominates the sequential timer at that scale, making that explicit ratio an experimental outlier rather than an indicator of pure compute efficiency.

6.3. The 1024-Bit Peak and Bandwidth Decline

For the 1024-bit workload, speedup rises from the lower occupancy points up to a peak of $30.5\times$ at $N = 1\text{K}$ before experiencing a performance decline down to $21.4\times$ at $N = 100\text{K}$ (as illustrated in Figure 1 and panel (a) of Figure 2). The initial performance ascent reflects the improving SM occupancy achieved as larger problem sizes fill the hardware execution pipelines across the 16 SMs.

The subsequent performance degradation beyond $N = 1\text{K}$ marks the transition of the execution envelope into

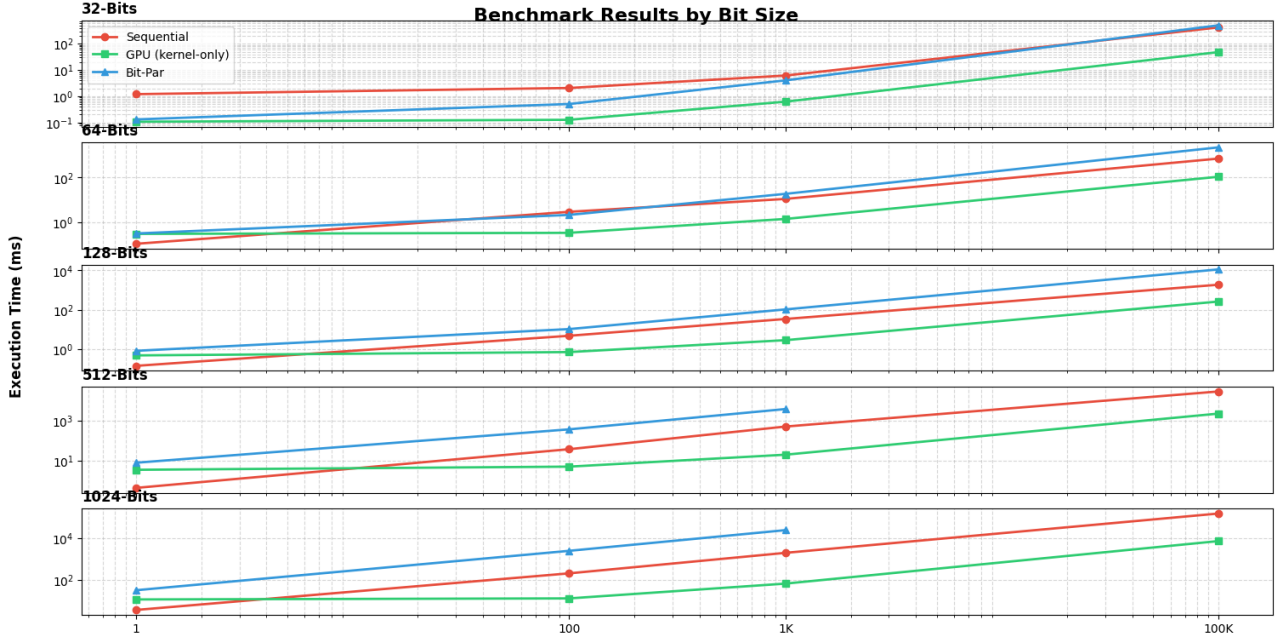


Figure 2: (a) 1024-bit kernel runtime on a log-log scale for all three implementations. The double-headed arrows show the $30.5\times$ GPU-over-sequential and $380\times$ GPU-over-Bit-Parallel margins at $N = 1K$. The $\times$ marks the OOM failure of Bit-Parallel at $N = 100K$. (b) Slowdown factor T_{BP}/T_{GPU} at $N = 1K$ by bit-width (log y -scale). Slowdown grows from $6\times$ at 32-bit to $380\times$ at 1024-bit as the $O(\text{bits}^2)$ Phase 1 workload scales quadratically compared to linear GPU Batch compute scaling.

a memory-bandwidth-limited regime. Once the aggregate working data set grows past the capacity of the L2 cache, active warps encounter long stall cycles waiting on global memory reads rather than sustaining active arithmetic execution. The $30.5\times$ peak represents the ideal architectural equilibrium point balancing maximum warp occupancy with optimal cache locality.

6.4. Bit-Parallel

The Bit-Parallel implementation produces mathematically valid results across all non-failing evaluation cases. At 128-bit operand widths and above it is slower than the GMP sequential baseline at every tested batch size; at 32-bit and 64-bit widths a marginal advantage over the sequential baseline appears at small N due to GMP’s relatively larger adaptive-precision overhead, but this reverses as N grows. Against GPU Batch, Bit-Parallel falls behind by factors ranging from $6\times$ to $380\times$ (Figure 2b). This performance gap widens dramatically as a function of bit-width because the Phase 1 computational complexity scales quadratically ($O(\text{bits}^2)$), whereas the GPU Batch implementation scales linearly ($O(\text{bits})$) with respect to the exponent size.

At 1024 bits and $N = 1K$, the Bit-Parallel approach is roughly $380\times$ slower than GPU Batch (24888 ms versus 65.50 ms); at $N = 100K$, the massive space overhead

required for the precomputation phases completely exhausts the 4 GB VRAM allocation limit, triggering an OOM failure. A rigorous mathematical proof demonstrating why the Bit-Parallel formulation cannot outpace sequential execution even when assuming infinite Phase 1 parallelism is provided in Section 7.3.

7. Evaluation Metrics

Both implementations are analysed through five formal architectural metrics. All numerical evaluations use the empirical timings compiled in Tables 3 and 4 (extracted from the master dataset in Table 2) along with the following hardware constants: TPI = 32, BITS = 1024, 16 SMs $\times$ 128 cores/SM = 2,048 CUDA cores, and 48 resident warps/SM $\Rightarrow$ 768 concurrently resident warps. Let t_s denote the hardware execution time for a single 1024-bit CGBN modular multiplication or squaring operation, treated here as having equal cost.

7.1. Speedup

$$S = \frac{T_{\text{sequential}}}{T_{\text{parallel}}} \quad (1)$$

Here, $T_{\text{sequential}}$ represents the GMP-based CPU execution runtime (which includes cold-start file I/O op-

erations), while T_{parallel} represents the isolated CUDA kernel execution time (excluding PCIe transfer latencies). This structural asymmetry slightly overstates the actual GPU speedup at very small batch sizes (N), where file-opening latencies inflate $T_{\text{sequential}}$, but becomes negligible as N scales.

Table 3: GPU Batch Speedup at Selected Configurations

Bits	N	T_{seq} (ms) / T_{GPU} (ms)	S
32	100	2.066 / 0.127	16.3×
64	10K	83.38 / 10.64	7.8×
128	10K	360.0 / 24.70	14.6×
512	1K	501.7 / 20.20	24.8×
1024	1K	1998 / 65.50	30.5×
1024	100K	157848 / 7381	21.4×

Table 4: Bit-Parallel Speedup Relative to Sequential ($S < 1$)

Bits	N	T_{seq} (ms) / T_{BP} (ms)	S
32	100	2.066 / 0.507	4.07×
64	10K	83.38 / 224.4	0.371×
128	100K	1864 / 11082	0.168×
512	1K	501.7 / 3698	0.136×
1024	1K	1998 / 24888	0.0803×
1024	10K	20406 / 252814	0.0807×

Three primary architectural interactions govern the GPU Batch speedup profile. First, $S < 1$ at low values of N because of hardware under-utilization and pipeline latency stalls. Because measurements are strictly kernel-only, this deficit is not driven by host-side kernel launch overhead; rather, when the batch size is extremely small, the absolute number of active warps in flight is insufficient to hide the deep arithmetic instruction latencies inherent to the long, dependent square-and-multiply loop chain. The execution pipelines frequently stall waiting for dependent data registers to clear.

Second, S scales up rapidly with N as the increased volume of concurrent problem instances provides the hardware thread scheduler with enough active, independent warps to fully saturate the streaming multiprocessors (SMs) and conceal these pipeline latencies via concurrent execution. Third, S scales positively with operand bit-width; this occurs because CGBN’s fixed 1024-bit precision path execution overhead shrinks relative to the true active operand size. The performance decline from $30.5\times$ at $N = 1\text{K}$ down to $21.4\times$ at $N = 100\text{K}$ marks the system’s transition into a memory-

bandwidth-limited regime, analyzed via Amdahl’s Law below.

7.2. Work Efficiency

Work efficiency measures the fraction of total executed computational effort that is strictly necessary to produce the correct mathematical result. If W_{min} is the minimum algorithmic work required and W_{total} is the total work actually performed by the hardware:

$$E = \frac{W_{\text{min}}}{W_{\text{total}}} \quad (2)$$

GPU Batch. Every problem instance executes a complete square-and-multiply chain bounded by CGBN’s fixed internal bit-width ($\text{BITS} = 1024$). Assuming a random exponent with a 50% average Hamming weight, the execution loop contains BITS squarings and $\approx 0.5 \times \text{BITS}$ multiplications. All N instances execute this uniform chain layout. With $\text{TPI} = 32$ threads processing cooperatively, the total hardware work matches the algorithmic minimal bounds for this architecture:

$$W_{\text{total}} = W_{\text{min}} = N \times 1.5 \text{ BITS} \times 1,024 \text{ MACs} \quad (3)$$

All 32 threads inside every executing warp remain continuously active throughout the execution envelope. Once N is large enough to saturate the 16 SMs, the work efficiency reaches $E \approx 1.0$, achieving ideal GPU utilization.

Bit-Parallel Phase 1. Phase 1 spawns $N \times \text{BITS}$ independent instances. Instance i executes exactly i sequential squaring steps, which means the cumulative hardware work expands quadratically:

$$W_{\text{total}}^{\text{P1}} = N \sum_{i=0}^{\text{BITS}-1} i \cdot t_s = N \cdot \frac{\text{BITS}(\text{BITS} - 1)}{2} \cdot t_s \quad (4)$$

Conversely, the true minimum algorithmic work required to construct all BITS intermediate precomputed powers is simply $(\text{BITS} - 1)$ cumulative squarings per batch task, where each logical step squares its immediate predecessor:

$$W_{\text{min}}^{\text{P1}} = N \cdot (\text{BITS} - 1) \cdot t_s \quad (5)$$

This yields a formal Phase 1 work efficiency of:

$$E_{\text{P1}} = \frac{N \cdot (\text{BITS} - 1) \cdot t_s}{N \cdot \frac{\text{BITS}(\text{BITS} - 1)}{2} \cdot t_s} = \frac{2}{\text{BITS}} \quad (6)$$

Table 5: Phase 1 Work Efficiency $E_{P1} = 2/\text{BITS}$

Bit-width (Internal)	E_{P1}
32	6.25%
64	3.13%
128	1.56%
512	0.39%
1024	0.20%

An equivalent perspective comes from evaluating the instantaneous warp utilization. At any arbitrary squaring step t , instances residing in the subset $\{t, t+1, \dots, \text{BITS}-1\}$ remain active, while the completed instances in the subset $\{0, 1, \dots, t-1\}$ terminate, forcing their assigned warps to sit dormant. The instantaneous fraction of active instances at step t is given by $f_{\text{active}}(t) = (\text{BITS} - t)/\text{BITS}$. Averaging this metric across all $(\text{BITS} - 1)$ steps of the longest critical path chain yields:

$$\begin{aligned}
\bar{f}_{\text{active}} &= \frac{1}{\text{BITS} - 1} \sum_{t=0}^{\text{BITS}-2} \frac{\text{BITS} - t}{\text{BITS}} \\
&= \frac{1}{\text{BITS}(\text{BITS} - 1)} \sum_{t=0}^{\text{BITS}-2} (\text{BITS} - t) \\
&= \frac{1}{\text{BITS}(\text{BITS} - 1)} \cdot \frac{(\text{BITS} + 2)(\text{BITS} - 1)}{2} \\
&= \frac{\text{BITS} + 2}{2 \cdot \text{BITS}} = \frac{1}{2} + \frac{1}{\text{BITS}} \approx 50\%
\end{aligned}$$

Consequently, at any random execution moment during Phase 1 operations, approximately 50% of all allocated warps are completely idle. This represents a fundamental structural load-imbalance property of the parallelization layout that cannot be optimized via block scheduling without modifying the underlying algorithm.

Bit-Parallel Phase 2. All N instances iterate uniformly across all BITS positions; the compute load is execution-balanced, meaning $E = 1$ internally within Phase 2. However, the modular multiplication at each position is executed unconditionally to eliminate warp branch divergence. If the exponent bit is zero, the computed result is entirely discarded. For a random exponent profile with 50% average Hamming weight, half of Phase 2's total modular multiplication steps yield no net contribution to the final result, effectively wasting 50% of the consumed Phase 2 memory bandwidth.

7.3. Amdahl's Law

$$S_{\text{max}} = \frac{1}{f_s + \frac{1-f_s}{p}} \xrightarrow{p \rightarrow \infty} \frac{1}{f_s} \quad (7)$$

where f_s represents the irreducible sequential execution fraction and p is the number of concurrent parallel workers.

GPU Batch — Solving for f_s at Peak Speedup

For the GPU Batch implementation, individual tasks are fully isolated and independent. The sequential fraction f_s at the batch level is composed entirely of fixed per-batch overheads (such as driver initialization and warp scheduling setup). Because this overhead is invariant, it amortises as N scales up, driving $f_s \rightarrow 0$ and accelerating speedup at higher batch sizes.

At the peak empirically observed speedup configuration of $S = 30.5 \times$ (1024-bit operands, $N = 1\text{K}$), using $p = 768$ concurrently resident hardware warps, Amdahl's relation states:

$$30.5 = \frac{1}{f_s + \frac{1-f_s}{768}} \implies f_s + \frac{1-f_s}{768} = \frac{1}{30.5} \approx 0.032787$$

Extricating the sequential fraction yields:

$$\begin{aligned}
f_s \left(1 - \frac{1}{768}\right) &= 0.032787 - \frac{1}{768} \\
f_s \left(\frac{767}{768}\right) &= 0.032787 - 0.001302 = 0.031485 \\
f_s &= \frac{0.031485}{0.998698} \approx \mathbf{3.15\%}
\end{aligned}$$

The theoretical execution performance ceiling when assuming $p \rightarrow \infty$ equals:

$$S_{\text{max}}^{(p \rightarrow \infty)} = \frac{1}{0.0315} \approx 31.7 \times$$

At $N = 1\text{K}$, the system captures **96.2%** of its theoretical Amdahl maximum performance, proving that the implementation runs near its hardware limit. Conversely, at $N = 100\text{K}$, S drops back to $21.4 \times$, implying an effective serial fraction of $f_s \approx 1/21.4 \approx 4.7\%$. This artificial rise in f_s is driven by memory-bandwidth saturation: global memory stall cycles force execution warps to idle, structurally mimicking the behavioral profile of an un-parallelized serial bottleneck.

Bit-Parallel — Proving $S_{\text{max}}^{BP} < 1$

A major architectural consequence of Amdahl's Law is that the Bit-Parallel formulation cannot outperform the sequential binary exponentiation baseline, regardless of the volume of Phase 1 parallelism. This proof relies on treating the Phase 2 sequential scan as an irreducible serial component.

Sequential execution time (standard binary exponentiation with a 50% exponent Hamming weight):

$$T_{\text{seq}} = \left(\text{BITS} + \frac{\text{BITS}}{2} \right) \cdot t_s = 1.5 \text{ BITS} \cdot t_s \quad (8)$$

Minimum Bit-Parallel execution time with $p \rightarrow \infty$

Phase 1 workers: The Phase 1 critical execution path (the longest dependent instance chain) requires:

$$T_{\text{P1}}^{\min} = (\text{BITS} - 1) \cdot t_s \quad (9)$$

Phase 2 must execute sequentially within each tracked instance. To avoid warp divergence, the kernel runs all BITS modular multiplications unconditionally:

$$T_{\text{P2}} = \text{BITS} \cdot t_s \quad (10)$$

Accumulating these phases yields the total minimum theoretical Bit-Parallel execution time:

$$\begin{aligned} T_{\text{BP}}^{\min} &= T_{\text{P1}}^{\min} + T_{\text{P2}} \\ &= (\text{BITS} - 1) t_s + \text{BITS} t_s \\ &= (2 \text{ BITS} - 1) t_s \end{aligned} \quad (11)$$

The maximum achievable theoretical speedup bounds reduce to:

$$S_{\text{max}}^{\text{BP}} = \frac{T_{\text{seq}}}{T_{\text{BP}}^{\min}} = \frac{1.5 \text{ BITS}}{2 \text{ BITS} - 1} \xrightarrow{\text{BITS} \rightarrow \infty} \frac{1.5}{2} = 0.75 \quad (12)$$

Note: This bound assumes the sequential baseline operates at equivalent arithmetic precision (i.e., t_s is the same for both). For narrow operands, GMP's adaptive precision reduces its effective t_s substantially relative to CGBN's fixed 1024-bit cost, which is why Bit-Parallel can briefly surpass the GMP baseline at 32-bit and 64-bit widths for small N (Table 2). The proof is fully applicable at 1024-bit operands, where both implementations operate at comparable precision.

Table 6: Theoretical Speedup Ceiling of Bit-Parallel Over Sequential ($p \rightarrow \infty$)

Bits	$1.5 \times \text{BITS}$	$2 \times \text{BITS} - 1$	$S_{\text{max}}^{\text{BP}}$
32	48	63	0.762
64	96	127	0.756
128	192	255	0.753
512	768	1023	0.751
1024	1536	2047	0.750

Phase 2 execution by itself requires $\text{BITS} \cdot t_s$ time, which instantly consumes $\frac{2}{3}$ of the entire sequential budget ($1.5 \text{ BITS} \cdot t_s$). When Phase 1's dependent critical path adds its unavoidable $(\text{BITS} - 1) \cdot t_s$ delay, the

overall execution time climbs to $\approx 2 \text{ BITS} \cdot t_s$, ensuring the total runtime always sits above the sequential baseline. The algorithm starts with an immediate 33-parallel processing can overcome.

7.4. Thread Divergence

Warp branch divergence occurs when separate threads within an identical 32-thread warp try to execute different conditional execution pathways, forcing the underlying hardware to serialize execution and stall threads assigned to the inactive branch.

GPU Batch. CGBN's structural choice of $\text{TPI} = 32$ maps exactly one independent modular exponentiation task instance directly to one hardware warp. This means all 32 threads inside the warp co-operate on processing the same message. Consequently, every conditional control branch evaluated inside the square-and-multiply sequence resolves identically for all 32 threads in that warp. This includes the data-dependent `if (exp & 1)` multiplication branch that fires on roughly 50 iterations; the entire warp takes this path uniformly. This guarantees that both inter-instance and intra-instance branch divergence remain at strictly 0

Bit-Parallel Phase 1. All spawned Phase 1 instances run an unconditional squaring loop completely devoid of data-dependent conditional branches. All 32 threads per warp trace identical control flow, ensuring 0

Bit-Parallel Phase 2. Phase 2 contains a conditional status check to handle accumulator registration. While different problem instances maintain unique exponent bit patterns, the $\text{TPI} = 32$ mapping ensures all 32 threads inside a Phase 2 warp process the exact same instance, resolving the branch uniformly. The Phase 2 kernel completely bypasses runtime branching by unconditionally executing the modular multiplication and applying a clean `cgbn_set(acc, old_acc)` primitive to revert the state if the exponent bit is zero. This converts a potential branch stall into predictable, redundant arithmetic. All threads stay active at the cost of wasting $\approx 50\%$ of Phase 2 arithmetic operations, aligning with the work efficiency metrics in Section 7.2.

7.5. CGMA Ratio

The Compute-to-Global-Memory-Access (CGMA) ratio measures the volume of active integer arithmetic operations executed per byte of global memory traffic:

$$\text{CGMA} = \frac{\text{Integer MAC Operations}}{\text{Global Memory Traffic (Bytes)}} \quad (13)$$

High CGMA metrics indicate excellent utilization of the memory subsystem; the GPU spends its cycles com-

puting rather than stalling on memory buses. Each 1024-bit CGBN modular multiplication step is estimated to require $\text{TPI}^2 = 32^2 = 1,024$ integer multiply-accumulate (MAC) operations across the 32 cooperating warp threads.

GPU Batch

Each separate instance accesses global memory exactly once to load three 128 B vectors (base, exponent, modulus) and output one 128 B result vector, consuming 512 B of total global memory traffic. Working against this 512 B base, the instance executes a fixed $1.5 \times \text{BITS}$ modular multiplications entirely inside registers and warp-shuffle registers:

$$\text{CGMA}_{\text{Batch}} = \frac{1.5 \times \text{BITS} \times 1,024 \text{ MACs}}{512 \text{ B}} = 3 \times \text{BITS} \quad (14)$$

Because CGBN runs its internal execution loop at 1024-bit precision regardless of true input size, the arithmetic operations remain fixed while the actual work scaling varies with the problem’s mathematical scale:

Table 7: GPU Batch CGMA by Bit-Width (CGMA = $3 \times \text{ebits}$)

Bits	Big-muls	MACs	CGMA
32	48	49,152	96
64	96	98,304	192
128	192	196,608	384
512	768	786,432	1,536
1024	1,536	1,572,864	3,072

At 32-bit inputs CGMA = 96: the warp fetches 512 B and performs only 49,152 MACs — the 1024-bit CGBN precision overhead wastes arithmetic on unused limbs. At 1024-bit CGMA = 3,072: the same 512 B now fuels 1,572,864 MACs. This $32\times$ increase in CGMA from 32-bit to 1024-bit is the direct mechanical cause of GPU speedup growing with bit-width: higher CGMA means the GPU’s compute throughput advantage over a single CPU core is more fully realised.

Bit-Parallel Phase 1

Each individual Phase 1 instance i reads $2 \times 128 \text{ B}$ (base + modulus) and writes out $1 \times 128 \text{ B}$ (the computed power), yielding 384 B of traffic per instance. Because instance i executes exactly i squarings, its isolated CGMA ratio scales as:

$$\text{CGMA}_{\text{P1}}(i) = \frac{i \times 1,024 \text{ MACs}}{384 \text{ B}} \quad (15)$$

Instance 0 yields a CGMA of 0 (moving 384 B without calculating arithmetic), while Instance 1023 reaches a

peak CGMA of $(1023 \times 1024)/384 = 2,728$. Averaging this distribution across all BITS instances results in:

$$\begin{aligned} \text{CGMA}_{\text{P1}} &= \frac{1}{\text{BITS}} \sum_{i=0}^{\text{BITS}-1} \frac{i \times 1,024}{384} \\ &= \frac{1,024}{384 \cdot \text{BITS}} \cdot \frac{\text{BITS}(\text{BITS} - 1)}{2} \\ &= \frac{1,024 \cdot (\text{BITS} - 1)}{768} \end{aligned}$$

For a standard internal execution path of BITS = 1024:

$$\text{CGMA}_{\text{P1}} = \frac{1,024 \times 1,023}{768} \approx \mathbf{1,364}$$

This translates to only 44.4% of the CGMA achieved by the GPU Batch implementation. The structural load-imbalance penalty is clear: early-terminating instances load data but perform minimal arithmetic, dragging down the average.

Bit-Parallel Phase 2

Phase 2 is forced to read back the entire precomputed power table from global memory for every instance. These table lookups dominate memory traffic:

$$\begin{aligned} \text{Memory}_{\text{P2}} &= (\text{BITS} \times 128 \text{ B}) + 256 \text{ B} \\ &\approx (1024 \times 128 \text{ B}) + 256 \text{ B} \\ &= 131,328 \text{ B} \\ &\approx 131,072 \text{ B per instance} \end{aligned} \quad (16)$$

Because this 131 KB table exceeds the physical limits of standard CUDA shared memory allocations (typically 48 KB per block), every power lookup must directly hit global memory. The cumulative compute per instance is $\text{BITS} \times 1,024 \text{ MACs} = 1,048,576 \text{ MACs}$, yielding:

$$\text{CGMA}_{\text{P2}} = \frac{1,048,576}{131,072} = \mathbf{8} \quad (17)$$

Table 8: CGMA Summary: GPU Batch vs. Bit-Parallel (1024-bit)

Component	Mem/inst (B)	CGMA
GPU Batch	512	3,072
Bit-Par Phase 1 (avg)	384	1,364
Bit-Par Phase 2	131,072	8

Phase 2’s CGMA of 8 is **384 $\times$ lower** than GPU Batch’s 3,072. For every byte fetched from the VRAM, Phase 2 executes only 8 MAC operations. This extremely low ratio leaves the 2,048 compute cores starved for data and idling on memory bus stalls. This explains why Phase 2 execution completely dominates total Bit-Parallel runtime at large problem scales (N).

8. Conclusion

This work evaluates two architecturally distinct strategies for accelerating modular exponentiation on modern graphics hardware, revealing a precise system-level relationship between exposed parallelism and total algorithmic work.

The **GPU Batch implementation succeeds** because its layout matches the natural concurrency of the problem space. Processing N independent cryptographic messages—each retaining its inherent $O(\log e)$ sequential execution chain—maps cleanly onto N isolated GPU warps without introducing extra arithmetic operations. Performance scales positively with batch size N (as launch overheads amortize) and with word size (as CGBN’s 1024-bit zero-padding overhead diminishes), achieving a peak performance improvement of $30.5\times$ for 1024-bit operands at $N = 1K$. The subsequently observed speedup degradation at $N = 100K$ is driven by L2 cache exhaustion and global memory bus saturation, representing a hard physical memory-bandwidth limit rather than an algorithmic defect.

Conversely, the **Bit-Parallel formulation fails** to compete with the sequential baseline. While the mathematical concept of isolating independent intermediate powers is correct, the chosen decomposition strategy introduces a severe $512\times$ arithmetic work blowup during Phase 1 for 1024-bit words, leaves up to 50% of allocated warps completely idle at any given moment, and incurs a strict Amdahl performance ceiling due to the unavoidable serialization of Phase 2. Modifying Phase 1 to compute incremental squarings would eliminate the quadratic work blowup and clear the VRAM memory footprint, but the algorithm would remain structurally non-viable due to the residual Phase 2 bottleneck.

The broader parallel computing principle is clear: restructuring an algorithm to expose massive hardware parallelism is counterproductive if the transformation introduces more total computational work than the underlying execution pipelines can recover via concurrent execution. Task-level parallelization layouts that preserve an algorithm’s natural granularity and introduce zero arithmetic overhead consistently outperform forced intra-task decompositions where the sequential core loops cannot be fundamentally altered.

References

- [1] NVlabs. *CGBN: Cooperative Groups Big Numbers for CUDA*. GitHub, 2020. <https://github.com/NVlabs/CGBN>
- [2] T. Granlund *et al.* *GNU Multiple Precision Arith-*

metic Library (GMP). <https://gmplib.org>

- [3] D. B. Kirk and W.-M. W. Hwu. *Programming Massively Parallel Processors: A Hands-On Approach*, 3rd ed. Morgan Kaufmann, 2016.
- [4] R. L. Rivest, A. Shamir, and L. Adleman. “A method for obtaining digital signatures and public-key cryptosystems.” *Commun. ACM*, vol. 21, no. 2, pp. 120–126, 1978.
- [5] NVIDIA Corporation. *CUDA C++ Programming Guide*, CUDA Toolkit 13.1, 2024.
- [6] P. L. Montgomery. “Modular multiplication without trial division.” *Math. Comp.*, vol. 44, no. 170, pp. 519–521, 1985.